# Type-checking: making sure programs make sense

## A friendly tour of simple types

CS 413

October 20, 2026

# A tiny puzzle

**This looks fine**

```
if 3 < 5 then 10 else 20
```

**This one?**

```
if 3 then false else 20
```

Can we catch the problem before running the program?

Type-checking answers questions like:

- Is the condition a boolean?
- Do both branches agree on a result type?
- Is a function given the kind of input it expects?

# What do types buy us?

- Catch certain mistakes early, with a useful location and explanation.
- Document how pieces of a program are meant to fit together.
- Let later stages make safe assumptions (for example, about operations).

**A type checker is not an oracle**

It does not prove that a program is useful, terminates, or matches the programmer's intent. It checks a particular set of consistency rules.

# Our little language

**Expressions**

$$e ::= n \mid \text{true} \mid \text{false} \mid x \mid e + e \mid e < e \\ \mid \text{if } e \text{ then } e \text{ else } e \mid \lambda x{:}T.e \mid e\ e$$

**Types**

$$T ::= \text{Int} \mid \text{Bool} \mid T \to T$$

Examples:

- $42 : \text{Int}$
- $\text{false} : \text{Bool}$
- $\text{Int} \to \text{Bool}$ is the type of functions from integers to booleans.

# A type is a promise about use

## Read the arrow left to right

$\text{Int} \to \text{Bool}$ means "give me an integer; I'll return a boolean."

## A typed function

$$\lambda(x : \text{Int}).\ x < 10 : \text{Int} \to \text{Bool}$$

The annotation says what the function expects; the body confirms what it returns.

- Function types associate to the right: $\text{Int} \to \text{Bool} \to \text{Int}$ means $\text{Int} \to (\text{Bool} \to \text{Int})$.

# Typing judgments: the checker's question

$$\Gamma \vdash e : T$$

Read: "under assumptions $\Gamma$, expression $e$ has type $T$."

- $\Gamma$ is the environment: a lookup table for variables.
- Example: $\Gamma = \{x : \mathsf{Int},\ f : \mathsf{Int} \to \mathsf{Bool}\}$.
- Then $\Gamma \vdash f\ x : \mathsf{Bool}$.

**Why keep assumptions around?**

When checking a function body, we need to remember the parameter's type. When checking a variable, we look it up.

# The easy rules

**Constants and variables**

$$\frac{}{\Gamma \vdash n : \mathsf{Int}} \qquad \frac{}{\Gamma \vdash \mathsf{true} : \mathsf{Bool}} \qquad \frac{x : T \in \Gamma}{\Gamma \vdash x : T}$$

**Addition**

$$\frac{\Gamma \vdash e_1 : \mathsf{Int} \quad \Gamma \vdash e_2 : \mathsf{Int}}{\Gamma \vdash e_1 + e_2 : \mathsf{Int}}$$

**Comparison**

$$\frac{\Gamma \vdash e_1 : \mathsf{Int} \quad \Gamma \vdash e_2 : \mathsf{Int}}{\Gamma \vdash e_1 < e_2 : \mathsf{Bool}}$$

**Conditional**

$$\frac{\Gamma \vdash c : \mathsf{Bool} \quad \Gamma \vdash a : T \quad \Gamma \vdash b : T}{\Gamma \vdash \mathsf{if}\ c\ \mathsf{then}\ a\ \mathsf{else}\ b : T}$$

The line reads "if the things above it check, the thing below it checks."

# Functions: introduce, then use

**Function introduction** (check a lambda):

$$\frac{\Gamma, x : S \vdash e : T}{\Gamma \vdash \lambda(x : S).e : S \to T}$$

Assume the parameter has type $S$, then check the body.

**Function application** (check a call):

$$\frac{\Gamma \vdash f : S \to T \quad \Gamma \vdash a : S}{\Gamma \vdash f\ a : T}$$

The argument type must match the input type; the result has the output type.

**Quick check**

If $f : \text{Int} \to \text{Bool}$, then $f\ 7 : \text{Bool}$ is fine, while $f$ false is not.

# Work one through: an integer test

Check $\lambda(x : \mathsf{Int}).\ x < 10$.

$$\dfrac{\dfrac{}{x : \mathsf{Int} \vdash x : \mathsf{Int}} \qquad \dfrac{}{x : \mathsf{Int} \vdash 10 : \mathsf{Int}}}{\dfrac{x : \mathsf{Int} \vdash x < 10 : \mathsf{Bool}}{\emptyset \vdash \lambda(x : \mathsf{Int}).\ x < 10 : \mathsf{Int} \to \mathsf{Bool}}}$$

- Add the assumption $x : \mathsf{Int}$ while checking the body.
- Both sides of $<$ are integers, so the body is boolean.
- The whole function therefore has type $\mathsf{Int} \to \mathsf{Bool}$.

# Work one through: call the function

Let $\mathit{small} = \lambda(x : \mathsf{Int}).\ x < 10$. What is $\mathit{small}\ 4$?

$$\frac{\emptyset \vdash \mathit{small} : \mathsf{Int} \to \mathsf{Bool} \qquad \emptyset \vdash 4 : \mathsf{Int}}{\emptyset \vdash \mathit{small}\ 4 : \mathsf{Bool}}$$

Now try *small* false:

**Mismatch**

The function expects an Int, but the argument has type Bool. Reject the call before execution.

# How a checker can do the work

For this language, checking follows the syntax tree recursively:

1. A literal has its built-in type; a variable uses the environment.
2. For $e_1 + e_2$, check both children are Int; return Int.
3. For a conditional, check a Bool condition and matching branch types.
4. For a function, extend the environment with its parameter type.
5. For a call, check the function type and the argument type agree.

## A useful mental model

The rules are a recipe for a recursive program: inspect the outermost expression, check its parts, and combine their types.

# When the recipe says “no”

### Bad condition

`if 3 then ... else ...` Expected Bool; got Int.

### Bad operator input

`true + 1` + requires two integers.

### Branches disagree

`if ready then 0 else false` One branch is Int, the other Bool.

### Bad function argument

`(fun (n:Int) => n+1) false` Expected Int; got Bool.

Good diagnostics point to the expression and explain the expected and actual types.

# A reassuring promise: progress (informally)

For a well-typed program in a carefully designed language:

- Evaluation does not get stuck trying to add a boolean to an integer.
- A function call does not apply a boolean as though it were a function.

**This is the type-safety idea**

Types rule out a useful class of runtime failures. The formal result is often called *type soundness*; its details depend on the language and its operations.

It does not rule out every runtime failure: division by zero, bad input, or nontermination may still be possible.

# Try it together

Assume $f : \text{Int} \to \text{Bool}$ and $n : \text{Int}$. Find the type, or the first reason it fails:

1. $f\ (n + 1)$
2. if $f\ n$ then $n + 2$ else $0$
3. if $n$ then true else false

# Try it together

Assume $f : \text{Int} \to \text{Bool}$ and $n : \text{Int}$. Find the type, or the first reason it fails:

1. $f\ (n + 1)$
2. if $f\ n$ then $n + 2$ else $0$
3. if $n$ then true else false

## Answers

1. Bool.
2. Int.
3. Rejected: condition $n$ has type Int, but needs Bool.

# Takeaways

- A type describes how an expression may be used.
- A typing judgment combines an expression, its environment, and its type.
- Each syntax form gets a small rule; checking is recursive.
- Function types connect the argument you provide to the result you get.
- Simple types catch real mistakes, while leaving plenty of interesting bugs for later.

Questions?